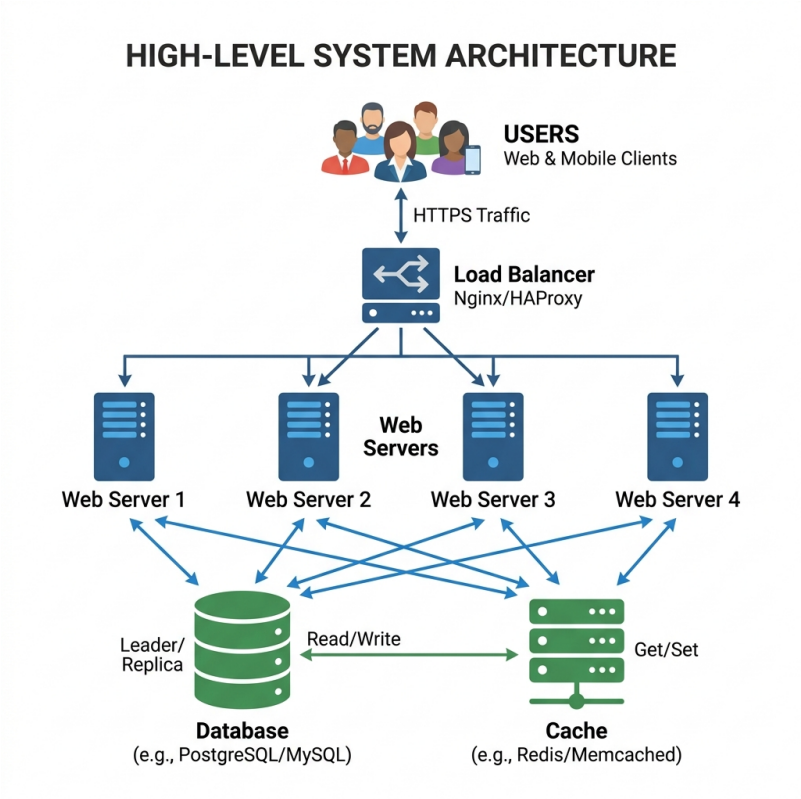
System Design for SDE Roles

The Ultimate Guide to Designing Scalable Systems



1. Introduction to System Design

System Design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. In software engineering interviews, it tests your ability to take a vague problem (e.g., "Design Twitter") and architect a scalable, robust, and efficient solution.

It bridges the gap between knowing how to code a feature and knowing how to serve that feature to 100 million concurrent users without the server crashing.

2. Scalability Basics

When a system receives more traffic than it can handle, it needs to scale. There are two primary ways to scale a system: Vertical Scaling (Scaling Up) and Horizontal Scaling (Scaling Out).

Vertical Scaling:

Adding more power (CPU, RAM) to your existing server. It is easy to implement but has a hard physical limit. You cannot add infinite RAM to a single machine, and it introduces a Single Point of Failure (SPOF).

Horizontal Scaling:

Adding more servers to your pool of resources. This is how massive systems like Google and Amazon scale. It requires more complex architecture, like Load Balancers, to distribute traffic evenly.

Load Balancing:

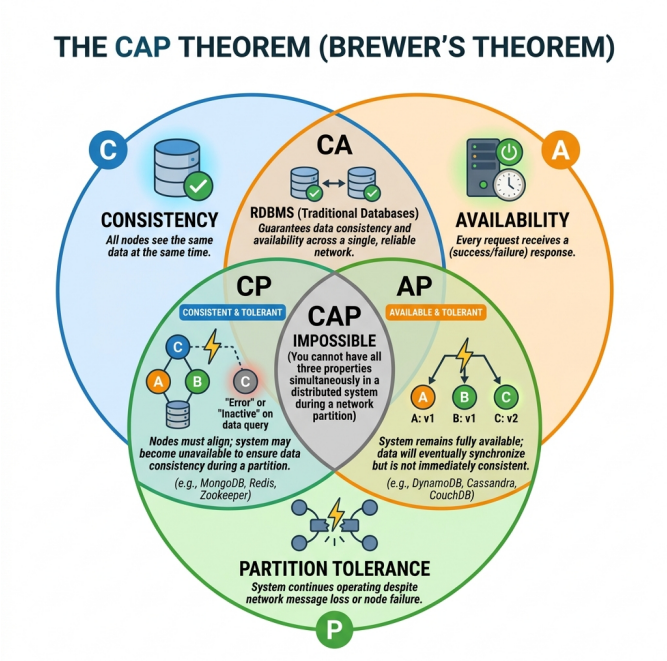
A Load Balancer acts as the "traffic cop" sitting in front of your servers and routing client requests across all servers capable of fulfilling those requests in a manner that maximizes speed and capacity utilization.

```
// A simple Nginx Load Balancer Configuration Example
http {
    upstream backend_servers {
        server 10.0.0.1;
        server 10.0.0.2;
        server 10.0.0.3;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://backend_servers;
        }
    }
}
```

3. The CAP Theorem

In theoretical computer science, the CAP theorem states that any distributed data store can only provide two of the following three guarantees simultaneously:



1. Consistency (C):

Every read receives the most recent write or an error. All nodes see the same data at the same time.

2. Availability (A):

Every request receives a (non-error) response, without the guarantee that it contains the most recent write.

3. Partition Tolerance (P):

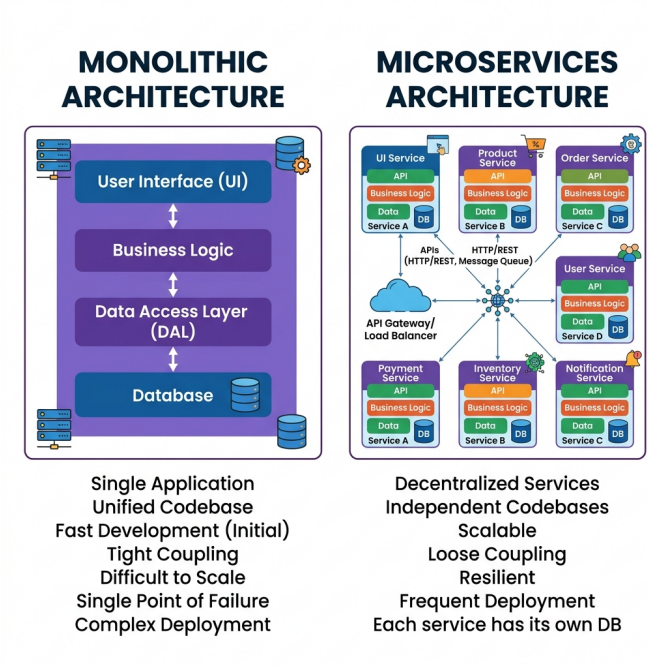
The system continues to operate despite an arbitrary number of messages being dropped (or delayed) by the network between nodes.

Interesting Fact:

The CAP Theorem was introduced by computer scientist Eric Brewer in 2000 as "Brewer's Conjecture". It was formally proven by Nancy Lynch and Seth Gilbert in 2002. In the real world, network partitions (P) are unavoidable, so engineers usually have to trade-off between Consistency and Availability (CP vs AP).

4. Microservices vs Monolith

A Monolithic architecture is built as one large system and is usually one codebase. Microservices architecture structures an application as a collection of loosely coupled, independently deployable services.



Caching for Performance:

A Cache is a short-term memory that has a limited amount of space, but is extremely fast. It contains the most recently accessed items. Systems like Redis or Memcached store data in RAM to prevent repeated expensive database queries.

Caching Strategies:

- Cache Aside: The application checks the cache first. If a miss occurs, it checks the DB and writes to the cache.
- Write-Through: Data is written into the cache and the corresponding database simultaneously.
- Write-Behind (Write-Back): Data is written to the cache alone, and writing to the DB is delayed until later.

5. Databases & Interview Blueprint

Choosing the right database is half the battle in System Design.

SQL vs NoSQL:

SQL (Relational): Best for structured data, strict schemas, and ACID compliance (e.g., Banking systems). Examples: PostgreSQL, MySQL.
NoSQL (Non-Relational): Best for unstructured/semi-structured data, rapid development, and easy horizontal scaling (e.g., Social media feeds). Examples: MongoDB, Cassandra, DynamoDB.

Database Sharding:

Sharding is a method for distributing data across multiple machines. It is a type of database partitioning that separates very large databases into smaller, faster, more easily managed parts called data shards.

The 5-Step System Design Interview Blueprint:

1. Requirements Clarification: (5 mins) Ask questions to define exact scope. Understand Read vs Write heavy.
2. Back-of-the-envelope Estimation: (5 mins) Estimate storage, bandwidth, and QPS (Queries Per Second).
3. High-Level Design: (10 mins) Draw the core components (Client -> LB -> Servers -> DB).
4. Deep Dive: (15 mins) Discuss specific components, trade-offs (e.g., choosing Cassandra over MySQL for high write throughput).
5. Identify Bottlenecks: (5 mins) Discuss SPOFs, how to handle massive spikes, and resolving caching issues.